

Actividad 4: Explotación y Mitigación de OAuth Inseguro

Tema: *Implementación segura de OAuth 2.0*

Objetivo: *Identificar errores en la autenticación OAuth y mitigarlos*

¿Qué es OAuth?

OAuth (Open Authorization) es un estándar abierto que permite a aplicaciones de terceros acceder a recursos protegidos en nombre de un usuario sin necesidad de compartir credenciales como contraseñas. **OAuth 2.0** es la versión más utilizada y se basa en la emisión de tokens de acceso para autenticar y autorizar a los usuarios.

Vulnerabilidad en la Implementación OAuth

Si OAuth 2.0 se configura incorrectamente, puede permitir varios tipos de ataques, como:

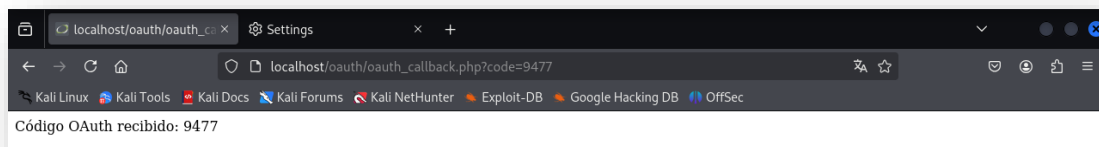
- **CSRF (Cross-Site Request Forgery):** Si no se usa un parámetro *state*, un atacante puede redirigir la autenticación a su propio código malicioso.
- **Token Leakage:** Si los tokens *se exponen en la URL o no se protegen adecuadamente*, un atacante podría reutilizarlos para acceder a los recursos protegidos.

Código Vulnerable (PHP)

Crear el archivo **vulnerable**: *oauth_callback.php*

```
<?php
if (isset($_GET['code'])) {
    echo "Código OAuth recibido: " . htmlspecialchars($_GET['code'], ENT_QUOTES, 'UTF-8');
} else {
    echo "Error: No se recibió ningún código OAuth.";
}
?>
```

El código NO valida NI almacena el estado (*state*) en la sesión, lo que lo hace vulnerable a ataques CSRF y además muestra el código *OAuth* directamente en la página, lo que permite que un atacante lo extraiga.



http://localhost/oauth/oauth_callback.php?code=9477

Explotar la Vulnerabilidad

1º Interceptar la Redirección OAuth

Un atacante puede manipular la URL de redirección para obtener el código de autorización.

2º Extraer y reutilizar el **code** en otro navegador

Si la implementación permite la reutilización del código, el atacante puede usarlo para obtener un token de acceso y autenticarse como la víctima.

3º Si el código se reutiliza, el sistema es vulnerable

Un atacante podría automatizar la extracción del código y usarlo para acceder a la cuenta de la víctima.

Mitigación y Solución Segura

* Usar el Parámetro *state* para Prevenir CSRF

Pasos para Configurar OAuth en Local:

1º Usar **localhost** o un Dominio Personalizado

Si tu proveedor OAuth permite **localhost como redirect_uri**, se puede usar:

```
http://localhost:8000/oauth_callback.php
```

Pero algunos proveedores **no permiten localhost directamente** por razones de seguridad. En ese caso, usa una de estas opciones:

- **127.0.0.1** en lugar de localhost:

```
http://127.0.0.1:8000/oauth_callback.php
```

- **Configurar un dominio local con */etc/hosts*** (solo en Linux/Mac):

1. Editar el archivo ***/etc/hosts*** y añadir:

```
127.0.0.1    miapp.local
```

2. Ahora se puede usar:

```
http://miapp.local:8000/oauth_callback.php
```

2º Iniciar un Servidor Local

Si estás usando PHP, iniciar un servidor local en el puerto 8000:

```
php -S localhost:8000 -t /var/www/html/oauth
```

Esto *servirá* la aplicación en:

```
http://localhost:8000/oauth
```

Opción 1: Usar un *Proveedor OAuth Real*

Si deseas probar OAuth en local con un proveedor como *Google*, *GitHub* o *cualquier otro*, se debe **registrar la aplicación en el proveedor OAuth** y configurar **redirect_uri** con *localhost*.

Ejemplo con *GitHub OAuth*

1. **Crea una aplicación en GitHub** desde: <https://docs.github.com/en/apps/oauth-apps>
<https://www.freecodecamp.org/news/how-to-set-up-a-github-oauth-application/>
2. **Configura redirect_uri en GitHub como:**

```
http://localhost/oauth/oauth_callback.php
```

3. **Usa la URL de autorización de GitHub en tu código:**

```
$auth_url = "https://github.com/login/oauth/authorize?" . http_build_query([
    "client_id" => "TU_CLIENT_ID",
    "redirect_uri" => "http://localhost/oauth/oauth_callback.php",
    "state" => $state,
    "scope" => "read:user"
]);
header("Location: " . $auth_url);
exit();
```

4. Ahora, cuando accedas a *http://localhost/oauth/oauth_login.php*, serás redirigido a la página de inicio de sesión de GitHub y, tras autenticación, **serás redirigido a tu entorno local**.

Opción 2: Montar un *Servidor OAuth en Local*

Si deseas **tener un servidor OAuth local**, se puede usar **Keycloak**, **OAuth2 Proxy**, o una implementación en **PHP** o **Node.js**.

Ejemplo con un Servidor OAuth en Local

Si deseas probar OAuth sin conectarte a un proveedor real, puedes usar **OAuth2 Proxy** con Docker:

1. **Instalar Docker:**

```
sudo apt update
sudo apt install apt-transport-https ca-certificates curl gnupg lsb-release -y
```

```
curl -fsSL https://download.docker.com/linux/debian/gpg | \
sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/docker-archive-keyring.gpg] \
https://download.docker.com/linux/debian bookworm stable" | \

sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin -y

sudo systemctl start docker
```

2. Ejecutar el servidor OAuth local:

```
sudo docker run -p 8080:8080 --name keycloak -e KEYCLOAK_ADMIN=admin -e
KEYCLOAK_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak start-dev

docker ps
```

3. Se puede usar como tu servidor OAuth.

Crear un Nuevo Realm en Keycloak

En Keycloak, un Realm es un espacio de autenticación y autorización que agrupa usuarios, roles y configuraciones de seguridad de una aplicación o conjunto de aplicaciones. Básicamente, un Realm es un entorno aislado donde puedes gestionar el acceso y las credenciales de los usuarios. Por lo tanto, permite gestionar múltiples aplicaciones dentro de un mismo servidor Keycloak sin interferencias entre ellas. Cada Realm tiene su propia configuración de autenticación, usuarios, roles y políticas de acceso y se pueden crear múltiples Realms para separar entornos (*por ejemplo, producción y desarrollo*).

Si nos disponemos de ello nos mostrará un error, por lo que se debe crear el realm *myrealm*:

Acceder a Keyloak através de la consola de administración:

```
http://localhost:8080/admin
```

Iniciar sesión con:

- **Usuario:** admin
- **Contraseña:** admin

Crear el Realm **myrealm**

1. En la parte superior izquierda, clic en el **menú desplegable** donde dice *Keycloak master*.
2. Seleccionar **"Create Realm"**.
3. **Nombre del Realm:** myrealm
4. Seleccionar **"Create"**.

Crear un Cliente OAuth para la Aplicación para definir roles y configurar autenticación.

Después de crear el realm *myrealm*, se necesita agregar un **cliente OAuth** para autenticar usuarios.

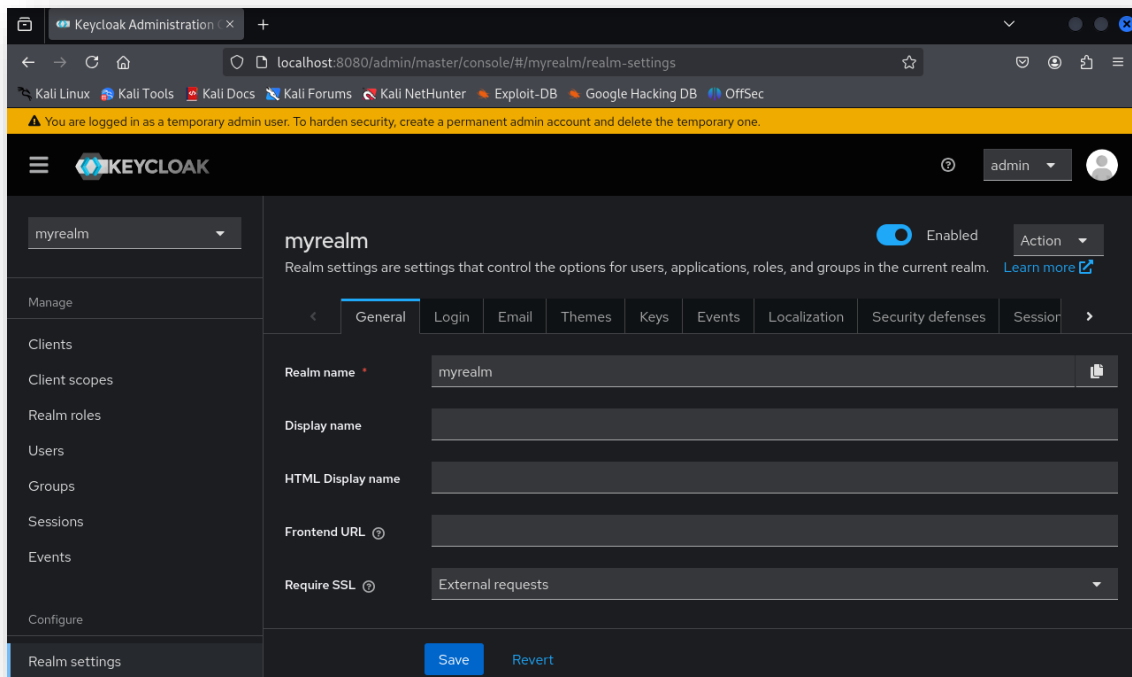
1. Entrar al realm **myrealm**.
2. Seleccionar "**Clients**" en el menú izquierdo.
3. Seleccionar "**Create**".
4. Configurar el cliente:
 - **Client ID:** myapp
 - **Client Protocol:** OpenID-Connect
 - **Access Type:** public
 - **Valid Redirect URIs:**

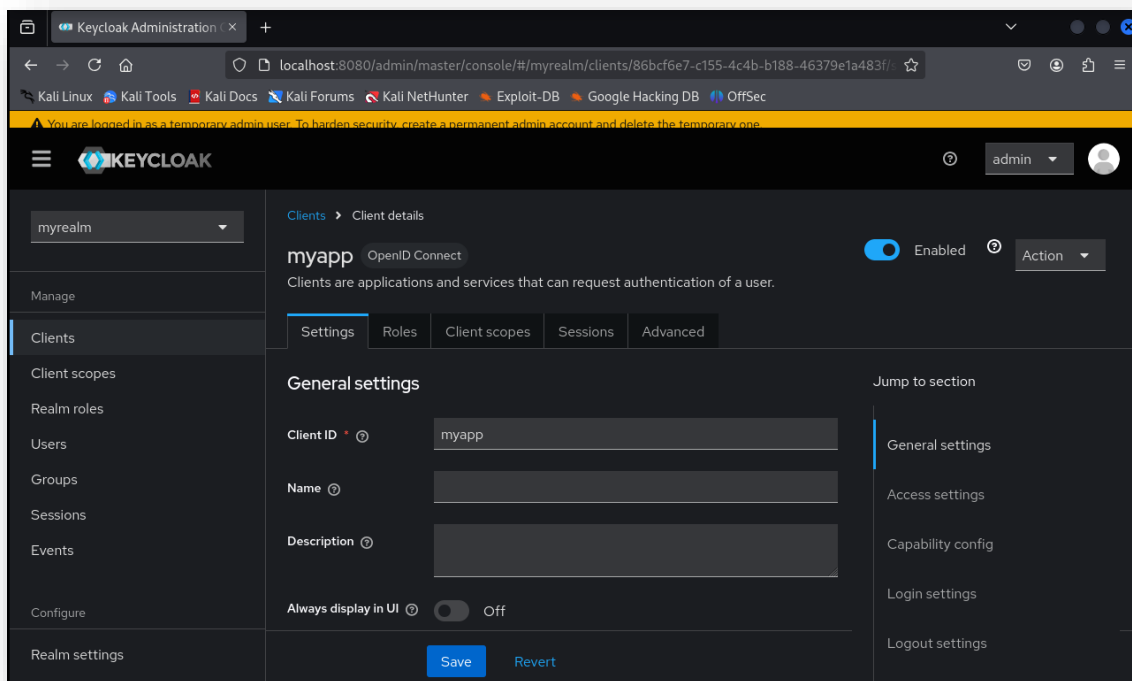
`http://localhost/oauth/oauth_callback.php`

- **Web Origins:**

`http://localhost/oauth`

5. Guardar los cambios.





Crear un usuario en **myrealm**

Si aún no se dispone de un usuario, se debe crear para iniciar sesión:

1. Ir a la *Consola de Administración*:

`http://localhost:8080/admin`

2. Seleccionar **myrealm**.

3. Sección "Users" (*Usuarios*) en el menú lateral.

4. Seleccionar "Add User" (*Agregar Usuario*).

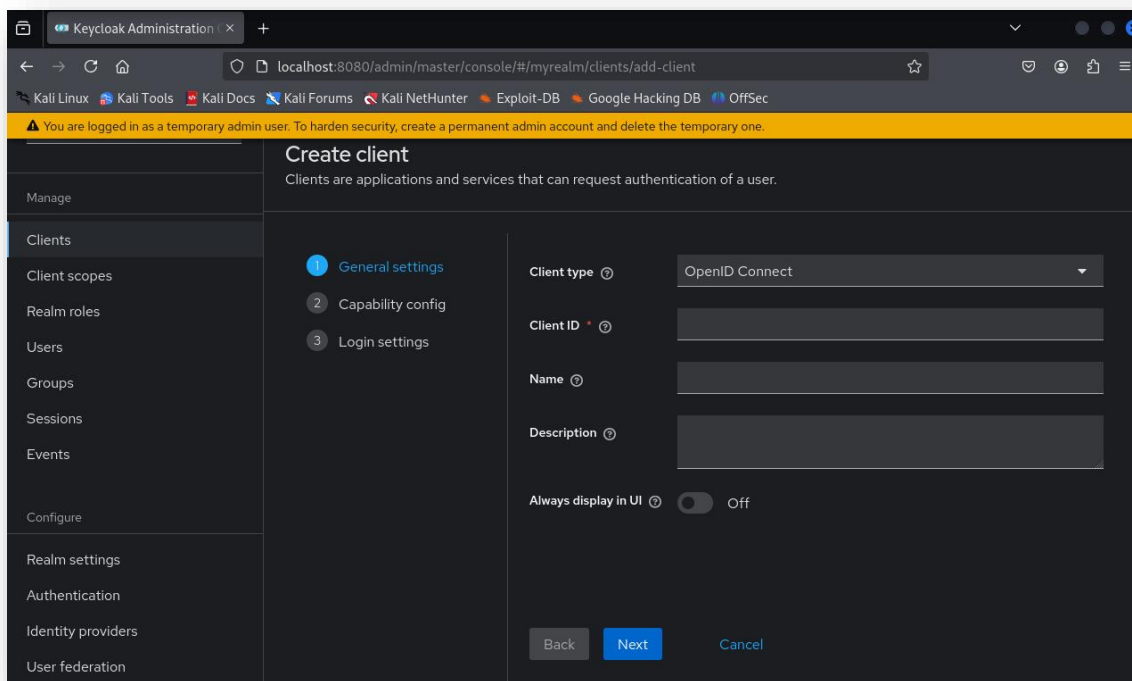
5. Rellenar los campos:

- Username: **testuser**
- Email: **testuser@example.com**
- First Name: **Test**
- Last Name: **User**
- Habilitar "Email Verified" (**opcional**)

6. Guardar el usuario.

7. Ir a la pestaña "Credentials" y establecer una contraseña (root).

Se puede usar este usuario para iniciar sesión.



3º Cambiar la URL de autorización en el código PHP:

```
$auth_url = "http://localhost:8080/realms/myrealm/protocol/openid-connect/auth?" .
http_build_query([
    "client_id" => "myapp",
    "redirect_uri" => "http://localhost/oauth/oauth_callback.php",
    "response_type" => "code",
    "state" => $state,
    "scope" => "openid"
]);
header("Location: " . $auth_url);
exit();
```

Para ello construimos la URL con localhost como *redirect_uri* en PHP:

Archivo final: **oauth_login.php**

```
<?php
session_start();

// Generar un token de estado aleatorio
$state = bin2hex(random_bytes(16));
$_SESSION['oauth_state'] = $state;

// Definir los parámetros OAuth
$params = [
    "response_type" => "code",
    "client_id" => "CLIENT_ID",
    "redirect_uri" => "http://localhost:8000/oauth_callback.php",
    "state" => $state
```

```
];

// Construir la URL de autenticación
$auth_url = "http://localhost:8080/realms/myrealm/protocol/openid-connect/auth?" . http_build_query([
    "client_id" => "myapp",
    "redirect_uri" => "http://localhost/oauth/oauth_callback.php",
    "response_type" => "code",
    "state" => $state,
    "scope" => "openid"
]);
echo "OAuth URL: " . $auth_url;
header("Location: " . $auth_url);
exit();

?>
```

4º Validar la respuesta en el *callback*

Archivo: **oauth_callback.php**

```
<?php
session_start();

// Verificar el estado (state) para evitar CSRF
if (!isset($_GET['state']) || $_GET['state'] !== $_SESSION['oauth_state']) {
    die("Error: Posible ataque CSRF.");
}
unset($_SESSION['oauth_state']); // Eliminar el `state` de la sesión

// Verificar si se recibió el código de autorización
if (!isset($_GET['code'])) {
    die("Error: No se recibió el código de autorización.");
}

$code = $_GET['code'];

// Datos de Keycloak
$client_id = "myapp";
// $client_secret = "TU_CLIENT_SECRET"; // Solo si es un cliente `confidential`
$redirect_uri = "http://localhost/oauth/oauth_callback.php";
$token_url = "http://localhost:8080/realms/myrealm/protocol/openid-connect/token";

// Configurar la solicitud cURL
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $token_url);
curl_setopt($ch, CURLOPT_POST, 1);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_HTTPHEADER, ["Content-Type: application/x-www-form-urlencoded"]);
curl_setopt($ch, CURLOPT_POSTFIELDS, http_build_query([
    'grant_type' => 'authorization_code',
    'client_id' => $client_id,
    // 'client_secret' => $client_secret, // Si es público, puedes omitirlo
    'redirect_uri' => $redirect_uri,
    'code' => $code
]));

// Ejecutar la solicitud y obtener la respuesta
$response = curl_exec($ch);
$http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
```


- Pueden ser almacenados en logs del navegador y servidores.
- Son visibles en el historial de navegación.
- Pueden filtrarse si se comparten enlaces accidentalmente.

Solución: Usar **POST** en lugar de **GET** para enviar tokens en el cuerpo de la solicitud.

Ejemplo incorrecto (evitar):

```
https://example.com/callback?access_token=eyJhbGciOi...
```

Ejemplo correcto (seguro): Solicitar el token con un POST seguro:

```
POST /token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&code=AUTH_CODE&client_id=CLIENT_ID&client_secret=CLIENT_SECRET
```

* Implementar PKCE para Aplicaciones Móviles

PKCE (*Proof Key for Code Exchange*) es una extensión de OAuth 2.0 que protege el *Authorization Code Flow* en clientes públicos, como aplicaciones móviles o SPA, evitando ataques de interceptación de códigos de autorización (MitM - Man in the Middle).

Las aplicaciones móviles no pueden almacenar de manera segura un *client_secret*, lo que las hace vulnerables a ataques de interceptación. PKCE soluciona esto agregando un código dinámico y verificable en cada solicitud de autorización.

Flujo de PKCE

1. Generación del **code_verifier** y **code_challenge** en la app móvil.
2. Solicitud de autorización con el **code_challenge** a la API de OAuth.
3. El usuario se autentica y recibe un **authorization_code**.
4. Intercambio del **authorization_code** por un **access_token**, enviando el **code_verifier**.
5. El servidor valida el **code_verifier** y emite un **access_token**.

```
$code_verifier = bin2hex(random_bytes(32));
$code_challenge = base64_encode(hash('sha256', $code_verifier, true));
```

Código Final

Inicio del Flujo OAuth con PKCE: *oauth_login.php*

```
<?php
session_start();

// Generar un `state` aleatorio para protección CSRF
$state = bin2hex(random_bytes(16));
$_SESSION['oauth_state'] = $state;

// Generar `code_verifier` y `code_challenge` para PKCE
```

```
$code_verifier = bin2hex(random_bytes(32));
$_SESSION['code_verifier'] = $code_verifier; // Guardar en sesión para validación posterior

$code_challenge = rtrim(strtr(base64_encode(hash('sha256', $code_verifier, true)), '+/', '-_'), '=');

// Construir la URL de autenticación con PKCE y `state`
$auth_url = "http://localhost:8080/realms/myrealm/protocol/openid-connect/auth?" .
http_build_query([
    "response_type" => "code",
    "client_id" => "myapp",
    "redirect_uri" => "http://localhost/oauth/oauth_callback.php",
    "state" => $state,
    "scope" => "openid email profile",
    "code_challenge" => $code_challenge,
    "code_challenge_method" => "S256" // Indica que se usa SHA-256 para PKCE
]);

// Redirigir a Keycloak
header("Location: " . $auth_url);
exit();
?>
```

Explicación de los Cambios en *oauth_login.php*

1. Generamos un *state* y lo guardamos en la sesión para prevenir ataques CSRF.
2. Implementamos PKCE:
 - o *code_verifier* es un valor aleatorio que guardamos en la sesión.
 - o *code_challenge* se calcula como un hash SHA-256 de *code_verifier* y se envía en la solicitud.
3. Keycloak almacenará *code_challenge* y lo comparará más tarde cuando se envíe *code_verifier* al obtener el token de acceso.

Intercambiar Código por Token con PKCE y POST: *oauth_callback.php*

```
<?php
session_start();

// Verificar el `state` para evitar CSRF
if (!isset($_GET['state']) || $_GET['state'] !== $_SESSION['oauth_state']) {
    die("Error: Posible ataque CSRF.");
}
unset($_SESSION['oauth_state']); // Eliminar el `state` de la sesión después de validarlo

// Verificar que se recibió el código de autorización
if (!isset($_GET['code'])) {
    die("Error: No se recibió el código de autorización.");
}

$code = $_GET['code'];

// Recuperar el `code_verifier` almacenado en la sesión para PKCE
if (!isset($_SESSION['code_verifier'])) {
    die("Error: PKCE `code_verifier` no encontrado en la sesión.");
}
$code_verifier = $_SESSION['code_verifier'];
unset($_SESSION['code_verifier']); // Limpiar la sesión después de usarlo

// Datos de Keycloak
$client_id = "myapp";
```

```
$redirect_uri = "http://localhost/oauth/oauth_callback.php";
$token_url = "http://localhost:8080/realms/myrealm/protocol/openid-connect/token";

// Usar cURL para enviar una solicitud POST segura
$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $token_url);
curl_setopt($ch, CURLOPT_POST, 1);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_HTTPHEADER, ["Content-Type: application/x-www-form-urlencoded"]);
curl_setopt($ch, CURLOPT_POSTFIELDS, http_build_query([
    'grant_type' => 'authorization_code',
    'client_id' => $client_id,
    'redirect_uri' => $redirect_uri,
    'code' => $code,
    'code_verifier' => $code_verifier // Se envía `code_verifier` para validar PKCE
]));

// Ejecutar la solicitud y obtener la respuesta
$response = curl_exec($ch);
$http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);

// Decodificar la respuesta JSON
$token_data = json_decode($response, true);

if ($http_code === 200 && isset($token_data['access_token'])) {
    echo "Token de acceso recibido:<br>";
    echo "<pre>" . htmlspecialchars(json_encode($token_data, JSON_PRETTY_PRINT), ENT_QUOTES, 'UTF-8') . "</pre>";
} else {
    echo "Error al obtener el token de acceso.<br>";
    echo "Respuesta del servidor: " . htmlspecialchars($response, ENT_QUOTES, 'UTF-8');
}
?>
```

Explicación de los cambios en *oauth_callback.php*

1. Verificamos *state* para prevenir ataques CSRF.
2. Recuperamos *code_verifier* de la sesión, que fue generado en *oauth_login.php*.
3. Intercambiamos el código por un token de acceso usando POST:
 - o Enviamos *code_verifier* junto con la solicitud, lo que Keycloak usará para validar la autenticación PKCE.
 - o No exponemos tokens en la URL, solo enviamos el código de autorización (*code*).
4. Usamos cURL para enviar la solicitud de forma más segura.

